

# EXECUTIVE SUMMARY

Target: http://bad-telemetry.com

Scan ID: TORTURE-P16-f47e9c8d

Scan Date: None

Scan Duration: -1

Total Requests Sent: not\_a\_number

Avg Latency: inf ms

Peak Concurrency: N/A

Findings: 2 vulnerabilities detected

Severity Breakdown: Critical: 1 | High: 0 | Medium: 1 | Low: 0

AI Inference Calls: 0

LLM Avg Latency: N/A

Circuit Breaker Activations: 0

- Detected 2 security issue(s) requiring attention.
- Immediate remediation recommended for critical findings.
- Review each finding below for detailed impact analysis.
- Prioritize fixes based on severity and exploitability.

## EXECUTIVE STRATEGIC IMPACT

Multiple vulnerabilities were identified that could potentially be chained for increased impact. Review each finding for cross-component risks.

# DETAILED FINDINGS

## FILTER: UNCATEGORIZED

### Finding #1: Cross-Site Request Forgery

HIGH

CWE: CWE-352  
CVSS Score: 7.5 (HIGH)

THREAT SCORE: 75/100

#### Description:

- Vulnerability 'Cross-Site Request Forgery' detected in target endpoint.

#### Impact:

- Unauthorized access to system resources confirmed.

#### Explanation:

Agent Gamma flagged this interaction based on high-confidence heuristic anomalies. Pattern 'CSRF' was intercepted to protect system integrity.

## FORENSIC ANALYSIS

Method: GET | Param: param\_0  
URL: http://test-target-torture-.com/api/v1/endpoint\_0  
Analysis: Insufficient input validation or output encoding on the server-side.

Table 1: Payload Decomposition

| Component    | Value          | Technical Function                            |
|--------------|----------------|-----------------------------------------------|
| param_0      | test_payload_0 | Direct reference to target resource.          |
| Access Check | Missing        | Application fails to verify authorization.    |
| Result       | 200 OK         | Server returns data for unauthorized request. |

Payload Specifications:

Vector Category: Cross-Site Request Forgery

Raw Payload: test\_payload\_0

Encoded: 746573745f7061796c6f616445f30

Encoding Type: None

Reproduction Command:

```
curl -X GET 'http://test-target-torture-.com/api/v1/endpoint_0'
```

HTTP Traffic Snapshot:

Request:

GET http://test-target-torture-.com/api/v1/endpoint\_0 HTTP/1.1

Host: target

{}

Response:

HTTP/1.1 200 OK

Content-Type: application/json

{"status": "vulnerable", "data": "revealed"}

Remediation:

- Implement strict input validation.

Recommended Code Fix:

```
# Remediation: Use parameterized queries and context-aware encoding.
```

FILTER: INJECTION & FUZZING

Finding #2: OS Command Injection

CRITICAL

CWE: CWE-78  
CVSS Score: 10.0 (CRITICAL)

THREAT SCORE: 100/100

Description:

- Vulnerability 'OS Command Injection' detected in target endpoint.

Impact:

- Unauthorized access to system resources confirmed.

Explanation:

Agent Gamma flagged this interaction based on high-confidence heuristic anomalies. Pattern 'COMMAND\_INJECTION' was intercepted to protect system integrity.

FORENSIC ANALYSIS

Method: PUT | Param: param\_1  
URL: http://test-target-torture-.com/api/v1/endpoint\_1  
Analysis: Insufficient input validation or output encoding on the server-side.

Table 1: Payload Decomposition

| Component    | Value          | Technical Function                            |
|--------------|----------------|-----------------------------------------------|
| param_1      | test_payload_1 | Direct reference to target resource.          |
| Access Check | Missing        | Application fails to verify authorization.    |
| Result       | 200 OK         | Server returns data for unauthorized request. |

Payload Specifications:

Vector Category: OS Command Injection  
Raw Payload: test\_payload\_1  
Encoded: 746573745f7061796c6f61645f31  
Encoding Type: None

Reproduction Command:

```
curl -X PUT 'http://test-target-torture-.com/api/v1/endpoint_1'
```

### HTTP Traffic Snapshot:

#### Request:

```
PUT http://test-target-torture-.com/api/v1/endpoint_1 HTTP/1.1
Host: target
{}
```

#### Response:

```
HTTP/1.1 200 OK
Content-Type: application/json

{"status": "vulnerable", "data": "revealed"}
```

### Remediation:

- Implement strict input validation.

### Recommended Code Fix:

```
# Remediation: Use parameterized queries and context-aware encoding.
```

# SCAN TIMELINE

[Orchestrator] TARGET\_ACQUIRED - 2026-03-04 17:10:18

[Sigma] JOB\_ASSIGNED - 2026-03-04 17:10:18

[Gamma] LOG - 2026-03-04 17:10:18

[Gamma] LOG - 2026-03-04 17:10:18

[Alpha] LOG - 2026-03-04 17:10:18

[Kappa] LOG - 2026-03-04 17:10:19

[Gamma] LOG - 2026-03-04 17:10:19

[Alpha] LOG - 2026-03-04 17:10:19

[Alpha] LOG - 2026-03-04 17:10:19

[Gamma] LOG - 2026-03-04 17:10:19

[Alpha] LOG - 2026-03-04 17:10:19

[Kappa] LOG - 2026-03-04 17:10:19

[Beta] LOG - 2026-03-04 17:10:19

[Gamma] LOG - 2026-03-04 17:10:19

[Beta] LOG - 2026-03-04 17:10:19

[Beta] LOG - 2026-03-04 17:10:20

[Kappa] LOG - 2026-03-04 17:10:20

[Kappa] LOG - 2026-03-04 17:10:20

[Alpha] LOG - 2026-03-04 17:10:20

[Beta] LOG - 2026-03-04 17:10:20

[Alpha] LOG - 2026-03-04 17:10:20

[Alpha] LOG - 2026-03-04 17:10:20

[Gamma] VULN\_CONFIRMED - 2026-03-04 17:10:20

[Gamma] VULN\_CONFIRMED - 2026-03-04 17:10:20

[Gamma] VULN\_CONFIRMED - 2026-03-04 17:10:20

[Kappa] GI5\_LOG - 2026-03-04 17:10:28